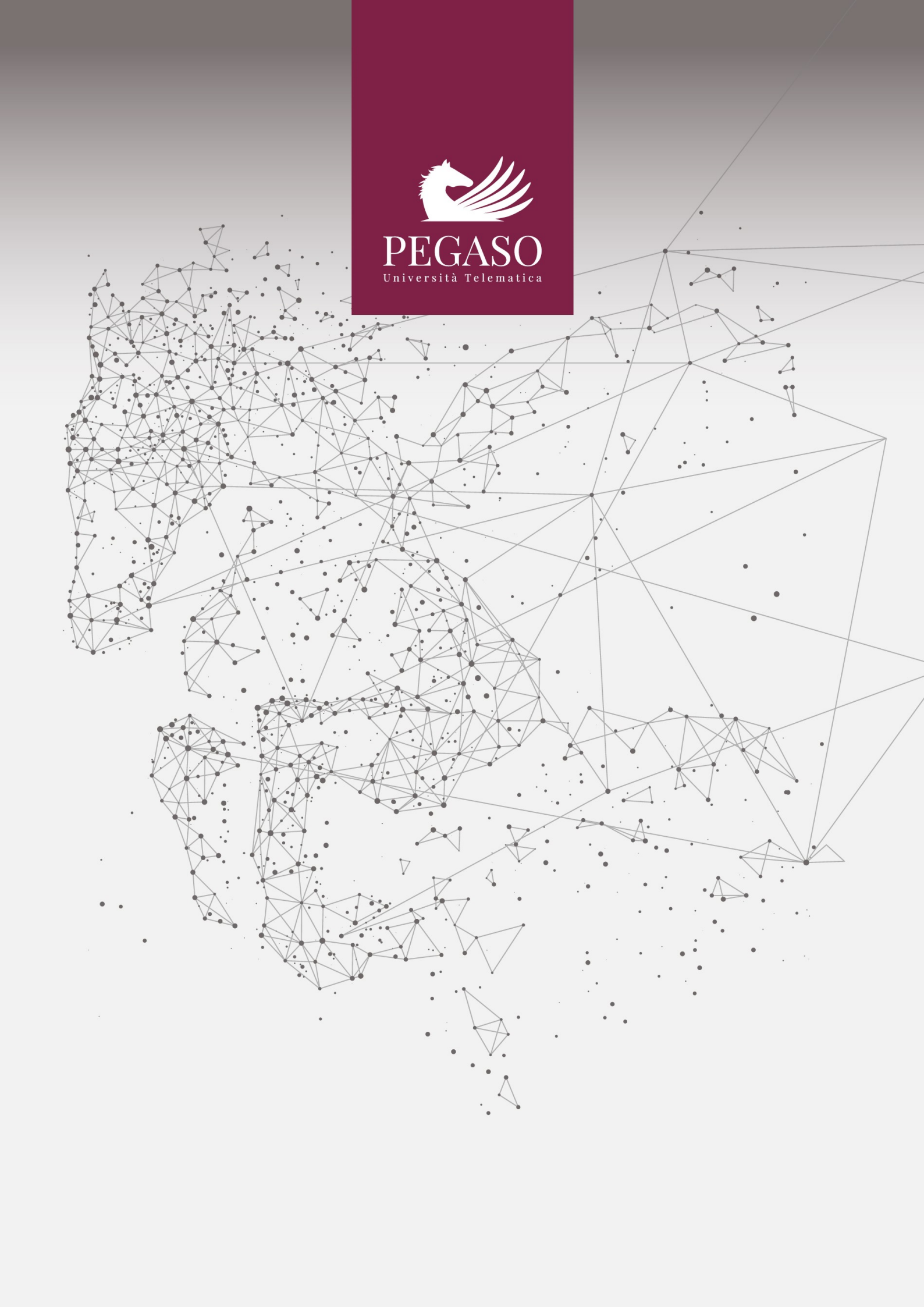

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. APPROCCIO DI SVILUPPO ORIENTATO AGLI OGGETTI.....3

BIBLIOGRAFIA 13

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Approccio di sviluppo orientato agli oggetti

1. Cos' è l'approccio orientato agli oggetti

Ci sono molti modi per affrontare un problema da risolvere con una soluzione software. La programmazione orientata agli oggetti rappresenta un modo di affrontare i problemi attraverso modelli la cui organizzazione è basata su concetti del mondo reale rappresentati dagli oggetti. Il dominio applicativo del problema è caratterizzato come un insieme di oggetti definiti come entità astratte che incapsulano specifici dati (attributi) ed operazioni su tali dati. Gli attributi specificano quello che l'oggetto è, le operazioni quello che l'oggetto fa (comportamento). Gli oggetti sono manipolati attraverso le operazioni che espongono e comunicano tra loro attraverso lo scambio di messaggi, ovvero invocazione reciproca delle operazioni.

La programmazione ad oggetti rappresenta, senza dubbio, il modello di programmazione più diffuso ed utilizzato degli ultimi trenta anni.

Le vecchie metodologie come la programmazione non strutturata e procedurale, in auge negli anni settanta, sono state lentamente ma inesorabilmente superate a causa degli innumerevoli vantaggi che sono derivati dall'utilizzo di questo nuovo paradigma di sviluppo consacratosi definitivamente negli anni '90.

2. Un po' di storia

La metodologia object-oriented fu proposta per la prima volta alla fine degli sessanta. Tuttavia, ci sono volute almeno vent'anni affinché la metodologia prendesse campo per diventare negli anni '90 il paradigma preferito per molti produttori di software e per un numero crescente di professionisti del software e dei sistemi informativi.

Le origini della programmazione ad oggetti si fanno risalire ai i primi linguaggi "ad oggetti" (SIMULA I e il SIMULA 67) sviluppati da Ole-Johan Dahl e Kristen Nygaard nei primi anni '60 in Norvegia. Entrambi questi linguaggi presentavano peculiarità che sono oggi tra i capisaldi della programmazione ad oggetti, come ad esempio le classi, le sottoclassi e le funzioni virtuali ma, a dispetto dell'importanza storica, tali linguaggi non riscossero particolare successo.

Negli anni '70 fu introdotto il linguaggio SmallTalk, coinsiderato da molti il primo vero linguaggio ad oggetti "puro", sviluppato in USA in due periodi successive inizialmente da Alan Kay, e successivamente fu ripreso da Adele Goldberg e Daniel Ingalls. Ma, ancora una volta, il grande successo tardò ad arrivare. Probabilmente, la ragione di ciò era da ricercare nel fatto che questo linguaggio (come il Simula) era

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

considerato, per lo più, uno strumento destinato alla ricerca e allo studio più che allo sviluppo. A questo si aggiunga che negli anni '70 il linguaggio C riscuoteva enorme successo grazie alle sue potenzialità e, soprattutto, al fatto che il famoso sistema operativo Unix (ancora oggi fortemente usato) fosse stato scritto utilizzando proprio tale linguaggio.

Fu insomma, necessario attendere gli anni '80 con l'avvento del linguaggio ADA per assistere al ritorno della programmazione ad oggetti come modello da utilizzare. Probabilmente la vera svolta fu rappresentata dall'avvento del C++, creato da Bjarne Stroustrup, nato negli anni ‘80 ma consolidantosi e definitivamente affermandosi negli anni ‘90.

Tra i più noti linguaggi di programmazione ad oggetti, citiamo il C++, Java, Delphi, C# e, come detto, il Visual Basic .NET.

Un ambito che più di altri riesce a sfruttare i vantaggi della programmazione ad oggetti è quello delle interfacce grafiche.

Recentemente, le tecnologie object oriented stanno rimpiazzando gli approcci classici di sviluppo software, quali la programmazione procedurale, e si stanno estendendo come approaccio alle diverse fasi di produzione del software, non solo quindi all’implemnetazoine del software tramite linguaggi object-oriented.

3. Programmazione non strutturata

La programmazione ad oggetti è andata lentamente ma inesorabilmente a soppiantare le vecchie metodologie come la programmazione non strutturata e procedurale, in auge negli anni sessanta e settanta.

Vediamo brevemente tali metodologie al fine di comprendere meglio le differenze con la programmazione ad oggetti.

Con la programmazione non strutturata il programma è costituito da un unico blocco di codice detto "main" dentro il quale vengono manipolati i dati in maniera totalmente sequenziale. Nella Fig. 1 viene rappresentata graficamente l'architettura di questo tipo di paradigma di sviluppo.



Fig. 1: Struttura di un programma con programmazione non strutturata

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d’autore (L. 22.04.1941/n. 633).

È importante notare che tutti i dati sono rappresentati soltanto da variabili di tipo globale, ovvero visibili da ogni parte del programma ed allocate in memoria per tutto il tempo che il programma stesso rimane in esecuzione.

È facile capire che un simile contesto è fortemente limitato e pieno di svantaggi. Ad esempio, sarà facile incappare in spezzoni di codice ridondanti o ripetuti che non faranno altro che rendere presto ingestibile ed "illeggibile" il codice, causando oltretutto un enorme spreco di risorse di sistema.

4. Programmazione procedurale

Un notevole passo in avanti, rispetto alla programmazione non strutturata, venne fatto con l'avvento della programmazione procedurale.

Il concetto base qui è quello di raggruppare i pezzi di programma ripetuti in porzioni di codice utilizzabili e richiamabili ogni volta che se ne presenti l'esigenza: nascevano le procedure. Ogni procedura può essere vista come un sottoprogramma che svolge una ben determinata funzione (ad esempio, il calcolo della radice quadrata di un numero, rappresentazione di un grafico a partire da dati) e che è visibile e richiamabile dal resto del codice.

Le procedure possono usare dati globali, oppure dati che vengono passati sottoforma di parametri di ingresso della funzione (per valore o per variabile). Possono poi far uso anche di variabili ad uso locale il cui valore viene perso quando si finisce l'esecuzione della procedura. L'interazione tra procedure avviene tramite condivisioni di dati.

Anche il flusso del programma è decisamente diverso rispetto a quello visto nella programmazione non strutturata: infatti, il main continua ad esistere ma al suo interno appaiono soltanto le invocazioni alle procedure definite nel programma. Quando una procedura ha terminato il suo compito il controllo ritorna nuovamente al main (o alla procedura che ne ha effettuato l'invocazione) che esegue una nuova chiamata ad un'altra procedura, fino alla terminazione del programma (vedi Fig.2)

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

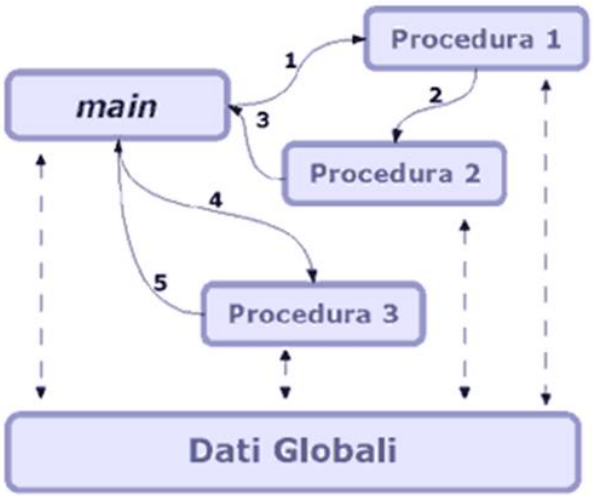


Fig. 2: Struttura di un programma con programmazione non strutturata

Il programma è quindi un fllusso di invocazioni a procedure il cui controllo resta nel main: si progettano le procedure e poi si stabilisce come queste debbano essere eseguite, decidendo il flusso di esecuzione e gestendo il relativo scambio di dati (tramite passaggio dati ai sottoprogrammi, uso dei valori di ritorno). Inoltre si stabilisce anche come debbano interagire tra loro le procedure attraverso la condivisione di dati globali.

I linguaggi più usati per questo tipo di tecnica sono Cobol, Fortran, Basic, C.

5. Principi di programmazione orientata agli oggetti

Per introdurre alla visione di sviluppo orientata agli oggetti, vediamo di arrivarci tramite un esempio. Consideriamo un'applicazione «acquisto di un vestito» e descriviamo come un software possa essere realizzato prima usando la programmazione di tipo procedurale e poi un approccio orientato agli oggetti per mettere a confronto i due approcci ed evidenziarne l'evoluzione.

Si consideri un cliente che entra in un negozio di abbigliamento, chiede al commesso un vestito di un certo tipo, colore e taglia (si optizzi per semplicità che il cliente ha già deciso quale vestito acquistare. Il commesso guarda in magazzino se è presente; in caso positivo comunica il prezzo; altrimenti l'applicazione si conclude senza acquisto. Se il cliente ritiene ragionevole il prezzo (confront ail prezzo con budget a disposizione) allora acquista l'abito altrimenti no e l'applicazione si conclude senza acquisto.

Un possibile sviluppo del programma in pseudocodice usando l'approccio procedurale è mostrato in Fig. 3 con le procedure indicate in rosso e i dati (variabili globali) indicate in blue.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
main ()
{
while (entrato_cliente == FALSE) attendi;
richiesta_vestito (tipo,colore,misura);
prezzo = controlla_disponibilità (tipo, misura, colore);
if (prezzo > budget)
       exit (NESSUNA_VENDITA);
else
   {
       vendi_vestito (tipo, misura, colore, prezzo);
       exit (VENDUTO);
   }
}
```

Fig. 3: Programma "Acquista vestito" usando programmazione procedurale

Il main consiste nell'invocazione delle seguenti procedure:

- attendi: sottoprogramma che gira finché non arriva un cliente (sarà imposterà una variabile globale entrato_cliente a TRUE appena entra un cliente così che il controllo ritorni al main).
- richiesta_vestito: accoglie ed elabora la richiesta del cliente
- controlla_disponibilità: controlla se il vestito è in magazzino (ritorna il prezzo se il vestito è disponibile, ritorna INT_MAX altrimenti)
- vendi_vestito: aggiorna le variabili in seguito alla vendita (decrementa la disponibilità in magazzino, aumenta l'incasso, etc.).

Vediamo adesso un possibile sviluppo usando l'approccio orientato agli oggetti, per il momento facendo uso di una notazione grafica intuitiva. Chiaramente, anche quanto mostrato può venire espresso in termini di codice facendo uso di particolare costrutti (che vedremo) messi a disposizione da un linguaggio di programmazione orientato agli oggetti. Va altresì detto che quanto mostrato potrebbe anche essere espresso tramite un diagrammi UML.

Nell'approccio orientato agli oggetti, si concentra l'attenzione sui dati in gioco nel problema e su come essi debbano interagire per raggiungere lo scopo del programma. In questo caso, i requisiti richiesti per il software è che un cliente deve poter entrare in un negozio, richiedere un vestito, decidere se comprare il vestito (se disponibile e se il budget è sufficiente) e in caso procedere al pagamento e all'aggiornamento del magazzino.

Come mostrato in Fig. 3, in questo esempio abbiamo un oggetto che rappresenta il cliente (che si riferisce alla cliente Carla) e un oggetto che rappresenta il negozio (che si riferisce al negozio Boutique

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Mara). Essi astraggono le proprietà salienti, rispettivamente, del cliente e del negozio nel contesto di questo problema.

Il cliente è identificato attraverso il suo budget e dalle informazioni relative al vestito scelto (tipo, colore, misura), mentre il negozio tramite il suo stato in_servizio (FALSE se inattivo o TRUE sta servendo il cliente), dall'incasso in un dato momento, e dal magazzino (inteso come vettore con le informazioni di ciascun prodotto incluso il prezzo e la disponibilità). Tali oggetti inoltre forniscono ciascuno anche le procedure per manipolare tali dati e per comunicare con esso nell'ambito dell'interazione con altri oggetti.

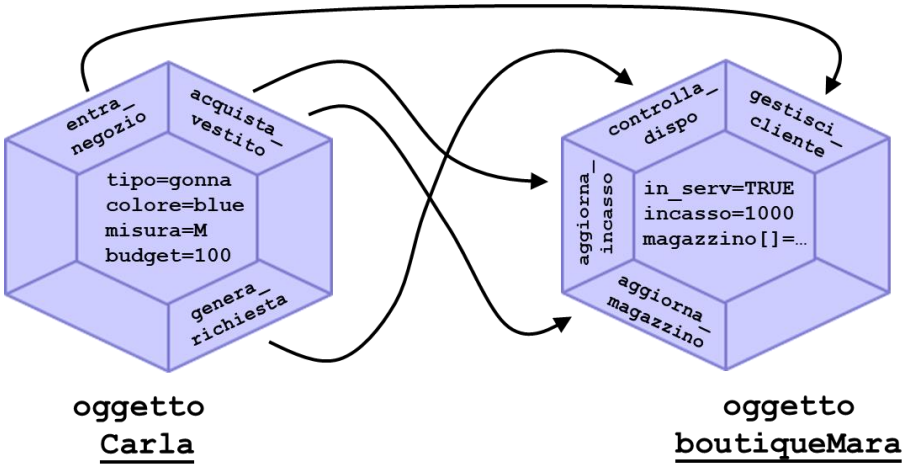


Fig. 3: Programma "Acquista vestito" usando i principi di programmazione ad oggetti

Il programma precedentemente espresso in modo procedurale può essere espresso tramite interazioni tra queste due oggetti invocando tali operazioni.

Infatti c'è bisogno che il cliente comunichi al negozio il suo arrivo. Per questo si prevede un'operazione per la classe cliente entra_negozio invocata all'arrivo del cliente dentro il negozio la quale va ad invocare l'operazione gestisci_cliente dell'oggetto negozio che pone lo stato del negozio in_serv a TRUE e mette l'oggetto in attesa della richiesta. Per la classe cliente si prevede anche un'operazione genera_richiesta con il quale il cliente comunica la scelta sul vestito desiderato e che a sua volta invoca controlla_dispo ritornando disponibilità e prezzo del vestito. Inoltre si un un'operazione acquista_vestito che in base al budget decide se procedere all'acquisto e se sì interagisce col negozio invocando le operazioni dell'oggetto negozio aggiorna_incasso e aggiorna_disponibilità per gli aggiornamenti relativi ad incasso e disponibilità.

Quindi in generale cosa possiamo dire?

Detto in termini più generali nella programmazione ad oggetti c'è un passaggio concettuale nella relazione tra dati e funzioni:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- nella programmazione procedurale sono definite le procedure e si impone loro di scambiare i dati.
- nel paradigma orientato agli oggetti si concentra l'attenzione sui dati e su come essi interagiscono tra di loro tramite opportune astrazioni.

L'oggetto è appunto l'entità che esprime un'astrazione sui i dati insieme alle operazioni possibili su quei dati, con operazioni e dati intrinsecamente specifici dell'oggetto e con un significato solo in relazione con il concetto associato ad esso.

Gli oggetti rappresentano entità dotate di un comportamento e che quindi possono svolgere operazioni, riportare e cambiare il loro stato e comunicare con gli altri oggetti del sistema, in generale hanno un comportamento.

Come mostrato in Fig. 4, l'applicazione è sviluppata costruendo la logica in termini interazioni tra gli oggetti realizzate attraverso invocazione delle operazioni (ovvero metodi) sui singoli oggetti strettamente connesse ai dati.

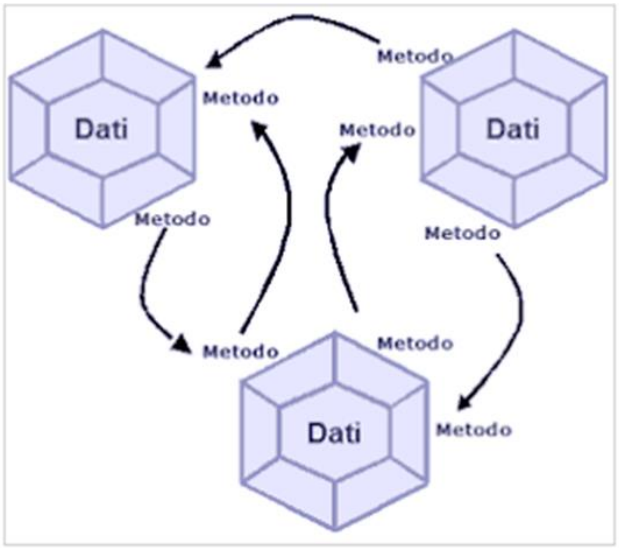


Fig. 4: Oggetti che interagiscono in un programma che usa l'approccio orientato ad oggetti

Un sistema complesso viene così visto come un insieme di oggetti interagenti ciascuna provvisto di una sua struttura dati e dall'insieme delle operazioni che l'oggetto sa effettuare su tale struttura.

Un oggetto comunica con un altro oggetto tramite dei messaggi. Un messaggio (con eventuali parametri) è una richiesta inviata ad un oggetto allo scopo di attivare delle operazioni in esso definite. Questo viene realizzato con una chiamata ad un metodo appartenente all'oggetto ricevente. In pratica a differenza di quanto avviene nei comuni linguaggi di programmazione il messaggio comunica all'oggetto cosa deve fare e non di come lo deve eseguire (è solo l'oggetto ricevente che conosce il modo nel quale deve reagire alla ricezione di un dato messaggio).

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La decomposizione di un problema in oggetti dipende dal criterio soggettivo e dalla natura del problema. Non esiste alcuna rappresentazione corretta in assoluto.

6. Concetti base: classi ed oggetti

Approfondiamo più nel dettaglio cosa si intende per oggetto e per classe.

L'**oggetto** è il costrutto con il quale si incapsulano i dati e le operazioni possibili su tali dati.

Un oggetto è un'entità definibile attraverso:

- Un insieme di dati (attributi o proprietà) i cui valori specificano lo stato nel quale si trova un oggetto
- Un insieme di operazioni (metodi) che sono l'interfaccia dell'oggetto verso l'esterno e che ne definiscono il suo comportamento.

Lo stato di un oggetto è rappresentato dai valori correnti dei dati o attributi.

Ad esempio, l'oggetto negozio è caratterizzato tramite il suo stato in servizio (se a TRUE vuol dire che ci sono clienti da servire), dal suo incasso in un dato istante, dallo schedario di magazzino. Inoltre il suo comportamento è specificato dalle operazioni: gestisci_cliente che pone lo stato del negozio in_serv a TRUE e mette l'oggetto in attesa della richiesta; controlla_disponibilità, che prende le richieste del cliente e le confronta con il dato magazzino (tabella di disponibilità e prezzi); aggiorna_incasso e aggiorna_dispo per gli aggiornamenti relativi ad incasso e disponibilità in magazzino.

Tuttavia bisogna ricordare che non sono gli oggetti, bensì le classi a essere descritti e specificati direttamente nell'approccio orientato agli oggetti.

Per trarre vantaggio dalla definizione di oggetto abbiamo bisogno di un sistema di classificazione degli oggetti. A questo scopo si definisce **classe** la specifica di una categoria di oggetti che condividono le stesse proprietà e lo stesso modello di comportamento (set di operazioni). Stesse proprietà è da intendersi da un punto di vista semantico, ovvero stesso set di attributi/proprietà anche se con valori che in generale assumeranno valori differenti nelle varie istanze dei singoli oggetti che saranno istanziate

La classe corrisponde grosso modo al concetto di tipo di dato negli altri linguaggi di programmazione, solo che è un po' più strutturato comprendendo anche le operazioni. La classe è come un calco da cui ottenere i singoli oggetti e il singolo oggetto creato è detto istanza della classe (i termini oggetto e istanza di classe sono usati come sinonimi) (vedi Fig. 5).

La scrittura del programma implica la definizione di un insieme di classi (tutte quelle che occorrono per rappresentare gli oggetti in gioco), la loro instanziazione e la codifica della logica di interazione per raggiungere lo scopo dell'applicazione.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

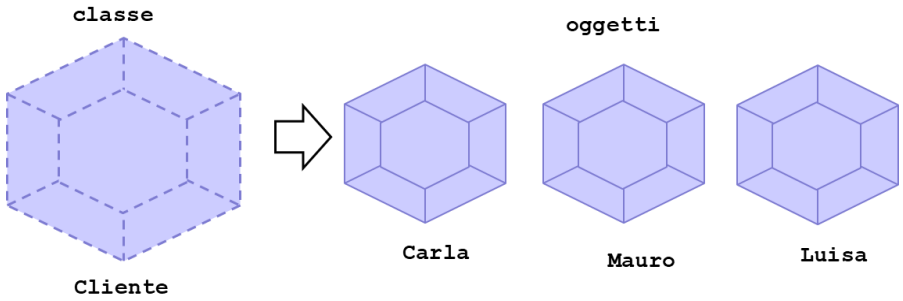


Fig. 5: Classe come "calco" da cui ottenere più istanze di oggetti che condividono semanticamente lo stesso set di dati ed operazioni.

Nello specificare una classe si fa un'operazione di generalizzazione ovvero si passa dal considerare pochi casi specifici a considerare una schiera di casi simili.

Riprendiamo l'esempio dell'acquisto di un vestito. Possiamo scrivere l'applicazione pensando a Carla che acquista alla Boutique Mara ma questo rendere l'applicazione cablata su un caso specifico. Quando si progetta un software ad oggetti le interazioni fra gli oggetti vanno pensate per il caso più generale possibile e ricondotto fra entità generalizzate le cui specifiche sono date appunto le classi da usarsi poi come modello per creare oggetti. A quel punto si può usare la classe per scrivere un'applicazione che comprende interazioni generiche fra client e negozi, sia che l'interazione avvenga tra Boutique Mara e Carla oppure tra Claudio e Centro Moda.

Come conseguenza del processo di generalizzazione, la specifica di una classe funziona come un modello (il calco apputno) che definisce attributi e operazioni che saranno condivisi da tutti le istanze create a partire da essa. Infatti come istanze della stessa classe, gli oggetti condividono la stessa semantica perché astrazioni dello stesso concetto. Per esempio, una volta creato l'oggetto Carla, volessimo definire un nuovo oggetto chiamato Mauro che è istanza della stessa classe Cliente, allora Mauro acquisisce tutti gli attributi di Cliente proprio come nel mondo reale, in cui sapendo che un oggetto è la realizzazione concreta di un certo concetto, automaticamente gli si attribuiscono tutte le proprietà usate per descrivere quel concetto.

Gli attributi degli oggetti che sono istanze della stessa classe presentano quindi la stessa semantica anche se in generale i valori nelle istanze saranno differenti. Tutte le istanze della classe Negozio saranno caratterizzati dall'attributo incasso per rappresentare il valore del ricavato in un certo giorno ma avrà in ciascuna istanza un valore diverso avendo essi realizzato in generale ricavati diversi in un dato giorno

L'astrazione del concetto che una classe rappresenta è definito anche in relazione al problema da risolvere. In altre parole, gli oggetti di una classe condividono oltre agli attributi e comportamento comuni,

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

anche un obiettivo semantico comune. Consideriamo ad esempio due entità quali una boutique (boutiqueMara) e un'auto (Fiat Panda). Se chi sviluppa li prende in considerazione (nell'ambito del dominio dell'applicazione) come beni finanziari ad esempio di una holding allora saranno modellati come due istanze diverse di una classe Asset, perché nell'ambito del problema da risolvere condividono un obiettivo semantico (rappresentare degli asset di una holding) e quindi condivideranno lo stesso set di attributi (es. valore, costo) seppur con valori in generale differenti. Se chi sviluppa li prende in considerazione (in base al problema da risolvere) come un'attività commerciale e come mezzo di spostamento allora questi concetti non condividono più lo stesso obiettivo semantico perché rappresentano concetti differenti e quindi saranno modellati da classi differenti perché sono la realizzazione concreta di due concetti differenti nel contesto del problema da risolvere.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- [1] R. S. Pressman, Principi di Ingegneria del software, McGraw-Hill, Quinta edizione
- [2] https://www.html.it/

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).